KG

PORTFOLIO

Kristopher Gallow

AI-native operator and systems builder. Senior Capital Underwriter with a Process Automation focus at Toast, plus four LLM-powered terminals shipped through Ghost Strategies LLC.

Live Site LinkedIn GitHub

7+	10	$500M+
Years in financial services	Production tools shipped	Capital deployments supported

FEATURED WORK

CM

COO/COLE Case Management Dashboard

Toast · 10-person underwriting team · 700+ active cases

Toast

Full-stack web app on a live Google Sheet (no backend) that replaced manual spreadsheet workflows. Real-time analytics scorecard (30-day, MoM, YTD, per-rep), Time-to-Decision buckets, a cohort-bucketed Bizops funnel, a per-rep diagnostics and count-audit panel, fraud and TCA tabs, plus an Address Change tab with a five-status lifecycle.

Stack Apps Script, JavaScript, Tailwind, Chart.js (2 custom plugins) **Users** 10

ET

Email Template Automation Suite

Toast · 14-person Capital Underwriting and Change of Ownership · 11 phases

Toast

Cross-platform desktop app that cut template retrieval from 30 seconds to under 3 seconds. Zero AI tokens consumed, zero customer data exposure by design. Includes a Salesforce-embedded NOIA generator concept and a stakeholder presentation.

Stack Python, Flask, PowerShell, AppleScript, PyInstaller, pystray **Platforms** macOS + Windows

FV

Funding Verification Tool

Toast · Capital Underwriting · 20-35 minute workflow cut to 40 seconds

Toast

Single-file HTML application that replaced a fragile multi-source VLOOKUP chain in Google Sheets. In-browser four-source JOIN engine with processor-specific match logic, manual overrides, audit labels, and a template-based 4-sheet XLSX export that preserves the master sheet's conditional formatting.

Stack Vanilla JavaScript, SheetJS, custom RFC 4180 parser **Sources** 4-way JOIN across 9 tabs

CX

CSV Column Extractor

Toast · Bizops · 9-minute workflow cut to under 55 seconds

Toast

Single-file HTML tool for non-technical operations users. Streams 1M+ row spreadsheets via parallel Web Workers, with a pre-read and pre-warm pipeline that takes the full open-to-paste workflow from approximately nine minutes to under 55 seconds.

Stack Vanilla JavaScript, Web Workers, Papa Parse, SheetJS, Promise.all **Scale** 1M+ rows/file

UC

Underwriting & Collections Toolkit

Toast · Capital Underwriting & Collections

Toast

A suite of zero-dependency, client-side tools that replaced brittle spreadsheet macros across lien review, application triage, and collections history — parse/de-dupe/classify UCC & tax-lien filings, get color-coded application-routing recommendations, and extract parent-account default & churn from a virtualized grid into paste-ready summaries, all in-browser with no data transmitted.

Stack React, Vanilla JS, Bookmarklets **Impact** Part of a toolset that cut review time ~60%

SQ

Self-Serve Snowflake Query Helper

Toast · Capital Underwriting

Toast

A teaching-oriented text-to-SQL tool that lets non-analysts draft schema-aware Snowflake queries in plain English — and learn the SQL as they go — removing the ad-hoc-query bottleneck on senior analysts. It never connects to Snowflake and sends only schema metadata; all LLM traffic routes through a server-side proxy that normalizes nine providers and keeps API keys out of the client. Runs as a shared team tool or personally with your own key.

Stack Vanilla JS, Cloudflare Workers, 9-provider LLM proxy **Modes** Team proxy or personal (BYO-key)

GS

Ghost Strategies Terminal Suite

Independent

Ghost Strategies LLC · LLM-powered financial tooling

Four production terminals built with Anthropic's Claude: Wealth Compass (client-side financial planning), Training Terminal (Python, SQL, JavaScript, and Rust learning terminal), Garage Terminal (asset tracking on FastAPI), and Economic Indicators Terminal (BLS, BEA, FRED integration).

Stack Python, FastAPI, JavaScript, Pyodide, sql.js

Discipline I own the logic, the LLM writes the code, I validate every output

T

TOAST INC · SENIOR CAPITAL UNDERWRITER, PROCESS AUTOMATION

Toast Work

Four production tools built for the Capital Underwriting and Change of Ownership teams. Each replaced a fragile or manual workflow with a self-contained, browser-based system.

COO/COLE Case Management Dashboard

Full-stack web app · 10-person underwriting team · 700+ active cases

A full-stack web application built on top of a live Google Sheet — no separate backend or hosting — that replaced manual spreadsheet workflows for case tracking, analytics, and address-change requests across a 10-person team and 700+ cases. The Cases tab tracks the full lifecycle with optimistic-UI edits written straight back to the sheet, the Address Change tab manages a parallel five-status workflow on its own sheet, and the Analytics tab is an operational scorecard with team-level, per-rep, and Bizops-funnel views.

MOCK VIEW

Case Management Dashboard · Cases (47 active · 3 picked today)

47	12	4.2	98%
Active Cases	Completed (7d)	Avg Days TTC	SLA Met

CASE #	RESTAURANT	SOURCE	STATUS	DAYS
C-1042	[Redacted]	Bizops	In Review	2
C-1041	[Redacted]	Collections	Picked	1
C-1040	[Redacted]	Direct	Completed	3
C-1039	[Redacted]	Bizops	In Review	5

KEY FEATURES

- Live case table backed by Google Sheets with an inline edit drawer, add/delete modals, and a manager Settings panel for self-serve config — plus a gated "Edit Case Details" mode that locks structural fields (Case #, Source, Type) behind an explicit toggle while status, rep, and notes stay instantly editable
- Auto-expires stale cases in place: "Info Gathering Sent" cases past 14 days flip to Expired, get date-stamped, and return corrected in the same read (a single flush fires only when a row actually changed)
- Analytics tab (30-day, MoM, YTD) with Time-to-Decision buckets and a Bizops/Collections funnel of three cohort-bucketed nested-bar charts (Created → Picked → Decided) that surface each month's open WIP as a visible band, separating queue-wait from rep turnaround
- Inherited-case days rebasing: for handoff cases, rep-time metrics are recomputed in-memory as (final date − pick date) so team and per-rep averages reflect rep-controlled time, not queue inflation — while system-time charts re-derive from raw dates and stay unaffected

- Rep Diagnostics panel: a per-rep deep dive (as submitter and as assigned rep) built to reconcile dashboard chart counts against sheet countifs, with row-level raw-value dumps that expose the hidden whitespace and casing that break exact-match counts
- Two custom Chart.js v4 plugins — barTotalsPlugin (per-bar and stacked totals) and nestedBarLabelsPlugin ("inner / outer" cohort labels) — with disciplined chart-instance teardown to avoid canvas-reuse leaks
- Hardened Sheets-as-database layer: a bottom-up last-row scan so appends never land at row 4,000+, LockService on concurrent writes, VLOOKUP formula copy + flush before read-back, UTC date normalization, and same-day 0-day clamping
- Address Change tab on its own sheet — five-status lifecycle, full add-request parity (mark Complete and "Checked in Clear" at creation), idempotent status-column and data-validation auto-creation, and unified GUID search across both workflows
- Idle-refresh banner after 30 minutes of inactivity and a version-update notifier that polls the deployed app version every 10 minutes

STACK

`Google Apps Script` `HTML` `CSS` `JavaScript` `Tailwind CSS` `Chart.js v4` `Google Sheets` `google.script.run` `LockService` `Google OAuth`

Email Template Automation Suite

11 phases · cross-platform desktop app · 14-person team · 30s to under 3s

A cross-platform desktop application that ships 20 pre-loaded email templates with rich-text formatting preserved on paste. Grew over 11 phases from a CLI tool to a full Flask web app, then to a PyInstaller-bundled native application with a system tray icon, and finally into a Salesforce-embedded NOIA generator concept presented to engineering stakeholders.

MOCK VIEW

Email Templates · 20 templates · 14-person team

Capital Underwriting	Received Response · Template ready · ● Copied
NOIA	NOIA MCA · {date} · {restaurant_name} · {source_of_debt}
Change of Ownership	COO DS Sent · {date} · {restaurant_name}

Retrieval: avg 2.1s · Tokens consumed: 0 · Customer data sent to model: 0

PHASE PROGRESSION

- Phases 1 to 3: CLI tool, cross-platform GUI launcher, then a full Flask web app
- Phase 4: Dark mode, brand logo, and accordion drawer system for the editor
- Phase 5: PyInstaller-bundled standalone executable with platform-appropriate data directories (macOS app support, Windows AppData), port discovery to dodge AirPlay on macOS, and a system tray icon for clean exits
- Phase 6: Salesforce-embedded NOIA template generator concept, with function-based template rendering (eight templates, conditional logic for variable-length entity lists)
- Phases 7 to 9: Placeholder fill modal with confirmation toasts, multi-line field support, two-panel preview modal, and product demo enhancements
- Phase 10: Template-sync bug fix (silent failure mode on every launch caught and resolved)
- Phase 11: Stakeholder presentation HTML with Apps Script architecture diagrams and a NOIA automation proposal for engineering review

ENGINEERING DECISIONS

- Zero AI tokens consumed by design: templates use placeholders that the user fills locally, so no customer data ever leaves the machine
- Cross-platform clipboard formatting: macOS uses AppleScript with the «class HTML» format; Windows uses CF_HTML byte-offset headers with precise UTF-8 byte counting
- 15KB Flask app deliverable instead of a 150 to 200MB Electron bundle, deliberately avoiding the Apple Developer Program signing requirement

- Per-machine identity: no authentication needed, each install is local-only

STACK

Python Flask Bash PowerShell JavaScript AppleScript Quill.js PyInstaller pystray Pillow

Funding Verification Tool

Single-file HTML · 20-35 minute workflow cut to 40 seconds

A single-file HTML application that replaced a fragile VLOOKUP chain in Google Sheets used to verify merchant banking data for capital deployments. Performs an in-browser JOIN across four sources (New Funding file, Snowflake export, Original CSV funding records, and DSL bank files), produces match results with processor-specific logic, supports real-time manual overrides, and exports a multi-sheet XLSX workbook from a bundled template that preserves the master sheet's conditional formatting. The tool grew to 9 tabs, including a TCA Check tab that auto-flags Toast Checking accounts and matches Slack-pasted GUIDs against an auto-detected canonical GUID column, and an External MCA Refi tab that tracks refi lineage between loans. The full funding verification workflow used to take 20 to 35 minutes per batch; the latest measured time (June 2026) clocked it at 40 seconds end-to-end.

BEFORE AND AFTER

BEFORE · 20-35 MINUTES

1. Manually maintain VLOOKUP chains across four Google Sheets
2. #N/A errors propagate when Snowflake data is delayed, no path to manual fix
3. No audit trail for manual SPA verifications or Toast Checking accounts
4. WORLDPAY and CHASE rules embedded in formulas, easily broken

AFTER · 40 SECONDS

1. Drag-drop four files, JOIN runs in milliseconds across 9 tabs
2. Manual Overrides tab surfaces Check Data rows, recompute on entry
3. Toast Checking and Verified in Merchant labels persist across re-uploads
4. Processor logic in code, 4-sheet XLSX export preserves master sheet formatting

MOCK VIEW

Funding Verification · Ops SPA Deals (12 rows)

EXT ID	PROC	MATCH R	MATCH A	LABEL
EXT-8821	WORLDPAY	YES	YES	Toast Checking
EXT-8820	CHASE	YES	Check Data	Pending
EXT-8819	WORLDPAY	NO	YES	Verified
EXT-8818	CHASE	YES	YES	Verified

KEY ENGINEERING DECISIONS

- Custom RFC 4180 CSV parser written from scratch to replace SheetJS's unreliable CSV mode (BOM stripping, mixed line endings, embedded quotes)
- MID normalization for JavaScript scientific notation: Snowflake XLSX exports return Merchant IDs as 1.23E+9, breaking direct string compares
- Hash-join via Map objects: equivalent to NewFunding LEFT JOIN Snowflake LEFT JOIN OriginalCsv LEFT JOIN DSL, executes in milliseconds for hundreds of thousands of rows
- Persistent DSL Worker Pool (size 3): operators routinely drop 4 to 10 large DSL files at once, so XLSX parsing runs through a fixed pool of three Web Workers fed by a queue and dispatcher, with zero-copy ArrayBuffer transfer and workers reused across drops rather than re-spawned
- Processor-specific match logic isolated in code: WORLDPAY requires exact DSL-to-file match; CHASE requires file value to differ from a known placeholder
- Real-time override injection: when a Snowflake lookup misses, manual GUID/MID/Proc Partner entry constructs a synthetic Snowflake-like object and the JOIN re-runs without a page refresh
- Labels and overrides keyed to External ID, survive re-uploads of any source file within the same session
- Template-based XLSX export: bundles the master Google Sheet's exact conditional formatting rules in a base64 template and edits the template XML at runtime via JSZip to inject the live data, preserving every YES/NO/Check Data cell fill from the original
- TCA Check subsystem: auto-detects the canonical Toast GUID column per source using a UUID-density times uniqueness score, auto-flags Toast Checking accounts from the Original CSV, and matches Slack-pasted GUIDs against in-batch records with per-loan and per-merchant dedup modes
- Refi lineage tracking: the External MCA Refi tab links each refi loan to its original (matching the External ID minus its suffix) and highlights the original loans magenta in the XLSX export
- Multi-sheet audit export: one workbook with the Ops sheet plus additional audit sheets, each filename-stamped MMDDYYYY-Funding Verification Tool.xlsx
- 9-tab structure: Ops SPA Deals (primary output with stat-pill filter shortcuts), four source upload tabs, TCA Check, Refi, External MCA Refi, and Manual Overrides
- Ships with two companion artifacts: a 13-page Standard Operating Procedure (built with a python-docx script, 11 embedded screenshots, data-flow diagram, troubleshooting guide, and a domain glossary), and a standalone Funding File Validator that round-trip-checks the exported file against the source before handoff

STACK

`Vanilla JavaScript` `SheetJS 0.18.5` `JSZip 3.10.1` `Custom RFC 4180 parser` `Web Worker pool` `Web FileReader API` `Base64 template embedding` `Clipboard API`

CSV Column Extractor

Single-file HTML · 9-minute workflow cut to under 55 seconds

A single-file HTML dashboard for combining up to five large spreadsheet files, each potentially exceeding one million rows, and extracting three specific columns for paste into a destination Google Sheet. Iterative performance engineering took the full open-to-paste workflow on three 25MB XLSX files from approximately nine minutes (manual baseline) to under 55 seconds.

BEFORE AND AFTER

BEFORE · ~9 MINUTES

1. Open three files sequentially in Excel or Sheets
2. Manually select and copy three columns from each file
3. Paste each chunk into the destination sheet, watch for cell-limit failures
4. Retry failed pastes after Google Sheets silently truncates

AFTER · UNDER 55 SECONDS

1. Drag-drop all three files into the tool
2. Pre-read and pre-warm pipeline starts in the background
3. Auto-parse fires 400ms after the last file, all three parse in parallel
4. Copy each column with one click, live cell-limit warnings prevent silent failures

MOCK VIEW

CSV Column Extractor · 3 files queued · auto-parse in 0.4s

847,302	3	~22s	3.9M
Total Rows	Files Parsed	Parse Time	Cells / Chunk

COLUMN	ROWS	CELL ESTIMATE	ACTION
Column B (MID)	847,302	22.0M cells	Chunk Required
Column L (R&T)	847,302	22.0M cells	Chunk Required
Column M (DDA)	847,302	22.0M cells	Chunk Required

Chunk size 150K rows · 6 chunks per column · safely under 10M cell limit

PERFORMANCE PIPELINE

- Parallel parsing via Promise.all: all files parse simultaneously in independent workers with local accumulators, merged in original file order after all Promises resolve

- Pre-read FileReader cache: readAsArrayBuffer fires the moment a file enters the queue, not when Parse is clicked, eliminating I/O wait from the critical path
- Pre-warmed worker pool: a Web Worker is spawned per XLSX file on drop, compiling SheetJS (about 500KB) inside the worker during the user's file-review window
- Auto-parse with 400ms debounce: no Parse button click required, handles single-file and multi-file drops uniformly, guards against re-triggering once results are displayed
- Selective XLSX cell extraction: the worker constructs only three required cell addresses per row (B+n, L+n, M+n) instead of allocating the full 2D array via sheet_to_json
- Batch and chunk sizes increased 4x: worker message batches went from 25K to 100K rows; CSV chunk size went from 5MB to 20MB, reducing postMessage round-trips proportionally

BASE ENGINEERING DECISIONS

- Papa Parse with worker: true for CSV streaming; hand-written Blob Worker for SheetJS XLSX parsing to keep the UI responsive during 1M+ row binary parses
- ArrayBuffer transferred via postMessage transferables to eliminate the memory cost of a second copy on large files
- 150K-row clipboard chunks: 150K rows times 26 default Google Sheets columns equals 3.9M cells, safely under the 10M limit
- Push-in-loop array construction instead of spread or push.apply, avoiding call-stack overflow on arrays exceeding 125K elements
- Three primary copy buttons (one per column) because the destination sheet uses non-adjacent columns; combined tab-separated copy preserved as a secondary option

STACK

`Vanilla JavaScript` `Web Workers` `Papa Parse 5.4.1` `SheetJS 0.18.5` `Blob Workers` `Promise.all parallelism` `FileReader API` `Clipboard API`

Underwriting & Collections Toolkit

Client-side underwriting & collections tools · React + vanilla-JS bookmarklets

A suite of zero-dependency, browser-based tools that replaced brittle spreadsheet macros across lien review, application triage, and collections history. Analysts paste or scrape third-party lien/UCC records (from a KYB data provider) and application details and get parsed, de-duplicated, recency-filtered, type-classified output — or a color-coded routing recommendation — to drop straight into notes. The same shared-engine pattern was extended to collections, extracting parent-account default and churn history from a virtualized data grid into a standardized, paste-ready summary. Everything runs locally, no backend, and transmits no data. Part of a set of underwriting tools that together cut review time by roughly 60%.

WHAT'S UNIQUE

- Lien Docket Dashboard: ingests pasted card text or raw API JSON, extracts per-filing fields, de-dupes, sorts newest-first, applies recency windows, and tags each filing by type with per-row badges so misclassification is obvious at a glance; summary metrics, diagnostics, one-click copy, and .txt/.csv export
- Two decision-support bookmarklets: a lien scraper that runs the identical parse/classify engine in-page against the logged-in provider (sidestepping cross-origin and authentication limits), and an App Router that reads live application fields and renders a color-coded review-depth recommendation
- Parent Default & Churn Extractor: reads parent-account default and churn history from a virtualized AG-Grid tracker into a standardized, paste-ready summary — defeating row- and column-virtualization by stitching cells into whole rows by stable column ID (not screen position), with a paste fallback that record-splits multi-line, JSON-valued cells; the grid was reverse-engineered first with a read-only DOM probe
- One shared parse/classify/format engine across every tool, so results never diverge; fail-loud by design — unreadable dates are kept and flagged, never silently dropped
- Config over hard-coding: keyword lists, recency windows, routing thresholds, and field labels live in editable config blocks so rule changes don't touch the logic

STACK

`React 18` `Babel Standalone` `Vanilla JavaScript` `Bookmarklets` `AG-Grid extraction` `Clipboard API` `Blob/URL export`

Self-Serve Snowflake Query Helper

Teaching-oriented text-to-SQL · single-file frontend + Cloudflare Worker proxy

A browser tool that lets non-analyst teammates draft accurate, schema-aware Snowflake queries in plain English — and learn the SQL as they go, through an explicit phrase-to-clause mapping. It removes the bottleneck of senior analysts writing every ad-hoc query while keeping execution in the user's own Snowflake session. By design it never connects to Snowflake and never sees a row of data: it handles only schema metadata (table and column names, types, comments) plus the user's generic natural-language ask. All model calls route through a Cloudflare Worker that holds provider keys server-side and normalizes nine LLM providers behind one contract.

WHAT'S UNIQUE

- Multi-provider proxy: one normalized request/response contract over nine LLM providers (Anthropic, OpenAI, Gemini, xAI, DeepSeek, Moonshot, Zhipu, Mistral, Qwen) via an adapter pattern — seven OpenAI-compatible plus dedicated Anthropic and Gemini adapters — resolving browser CORS and removing API keys from the client
- Personal or organizational: a first-run selector configures either an organizational mode (shared team Worker, provider keys as server-side secrets, gated by a Bearer token) or a personal mode — bring-your-own-key, calling Anthropic or Gemini directly from the browser, or your own proxy for all nine providers
- Self-updating model discovery: queries each provider's live model endpoint and merges it with a curated overlay (24-hour TTL), so newly released models appear automatically with no code edits — built on the insight that model names churn but API families stay stable
- One Low/Medium/High effort control mapped per model family to three distinct mechanisms (reasoning-effort enum, thinking-token budget, or omitted), and auto-disabled when the selected model can't reason
- Teaching layer: every result returns the SQL plus a plain-English explanation and a three-column phrase → clause → why mapping table, reframing a query generator into a learning tool
- Compliance-driven design: no database connection, metadata-only prompts, provider keys held as server-side secrets behind a token-gated, fail-closed proxy, and a client-side PII/MNPI pre-flight that warns on emails, phone numbers, GUIDs, and account-ID-shaped strings

STACK

`Vanilla JavaScript` `Cloudflare Workers` `Tailwind (CDN)` `Prism.js` `localStorage` `9-provider LLM proxy`

GS

GHOST STRATEGIES LLC · INDEPENDENT

Ghost Strategies

Four LLM-powered financial tools built independently. Every project ships with the same discipline: I own the logic and architecture, the LLM writes the code, I validate every output, and I only prompt with test or sample data so no proprietary information ever enters the model.

Wealth Compass Terminal

Client-side financial planning dashboard

A client-facing financial planning dashboard that parses a 13-sheet workbook with 755 formulas entirely client-side. All data stays on the user's device. No backend, no data transmission, no model exposure to client information.

STACK

`JavaScript` `HTML` `SheetJS` `Custom formula engine`

Training Terminal

Gamified four-language coding terminal · Python, SQL, JavaScript, Rust

A single-file, browser-based coding terminal (xterm.js) for practicing Python, SQL, JavaScript, and Rust. Runs real Python through Pyodide, real SQL through sql.js against a seeded relational database, and grades JavaScript inside a network- and storage-stripped Web Worker sandbox; Rust is graded by a literal-preserving structural matcher that accepts equivalent syntax. 200 curated questions — 50 per track across five gated tiers (Introductory, Amateur, Intermediate, Experienced, Master), 10 questions per gate — with localStorage-persisted progression and live tier and engine-status KPIs.

WHAT'S UNIQUE

- Four grading paths in one static file: Pyodide (Python), sql.js on a seeded auto-rebuilt database (SQL), an isolated Web Worker with network and storage APIs removed (JavaScript), and a structural normalizer with a syntax-equivalence system (Rust)
- Real xterm.js REPL: a command bar (start, hint, concepts, cheatsheet, examples, schema, reset, export/import), multi-line editing, command history, and emacs-style key bindings
- Learning layer baked in — per-tier concepts auto-surface on first entry, a toggleable cheatsheet panel, worked examples, and per-question hints
- 200 questions reviewed and calibrated by concept and difficulty; 10-question tier gates force mastery before the next rank unlocks, all persisted with no backend

STACK

`Vanilla JavaScript` `xterm.js` `Pyodide` `sql.js` `Web Workers` `localStorage`

Garage Terminal

Multi-asset tracker · Python FastAPI backend

A personal asset tracking terminal with a Python FastAPI backend and an automated data pipeline using GitHub Actions for weekly serverless CI/CD workflows. Pulls valuation data, refreshes the dashboard, and exposes a clean read API.

STACK

`Python` `FastAPI` `GitHub Actions` `JavaScript` `Serverless CI/CD`

Economic Indicators Terminal

BLS, BEA, FRED API integration

A macroeconomic data terminal that integrates with three public US government economic data sources (BLS, BEA, FRED) via a Python proxy. Surfaces unified indicators (inflation, employment, GDP, regional metrics) in a single dashboard.

STACK

`Python` `BLS API` `BEA API` `FRED API` `Proxy server`

AI-Assisted Financial Modeling Suite

Dynamic financial engines built with Claude

A separate workstream under Ghost Strategies focused on financial modeling rather than terminal apps. Includes a probability-weighted valuation model and a $3M TRS retirement planner. I own the financial logic and architecture while directing Claude to generate the code. Every output is validated against standard financial formulas before being trusted, and I only prompt with test or sample data to keep any real client information out of the model.

KG

ABOUT

Background and operating model

How I work and what I'm looking for.

I'm a Senior Capital Underwriter at Toast, where I support over $500M in active capital deployments and operate inside a fintech that scaled from $100M to over $2B in annual volume during my tenure. My day-to-day is capital decisions, customer health monitoring, and scenario planning, but my distinguishing work is building the systems and tools that operate alongside that core function. Ten of those tools are now in daily use across underwriting and collections, replacing manual workflows that used to consume hours of recurring team effort each week.

I'm part of Toast's Claude Code and Cowork rollout to a select group of process automation specialists. I built a multi-agent system with an orchestrator that routes work to specialized sub-agents (workflow automation, code audit, technical project management, devops, technical writing), each with its own system prompt, scoped tool access, and isolated context. The same architecture is replicated as Skills in Cowork.

Outside of Toast, I run Ghost Strategies LLC, where I ship LLM-powered financial tools. My operating discipline across every project is consistent: I own the logic and architecture, the LLM writes the code, I validate every output before I trust it, and I never feed proprietary data to a model.

I'm based in San Antonio, Texas, and I work remote-only. I'm targeting Senior Manager-track or Senior IC roles in strategy, operations, risk, or AI-process work at companies where the AI-native operating model is the default, not the exception.

CORE STACK

SQL (Snowflake) | Salesforce | Python | JavaScript | Google Apps Script | Hex | Excel and Google Sheets | Claude Code | Cowork | Multi-Agent Systems | Flask | FastAPI | Chart.js

KG

CONTACT

Get in touch

Best ways to reach me. I respond to email and LinkedIn within a business day.

DIRECT

EMAIL
krisgallow918@gmail.com

PHONE
832-233-9437

LINKEDIN
linkedin.com/in/kristophergallow

GITHUB
github.com/KaeJhee

LOCATION
San Antonio, TX (Remote)

SIDE PROJECT
kris@ghoststrategies.io

↓ Download resume (AI & Insights variant)